Banking Product & Pricing Backend Module

Academic & Enterprise Context: Derived from Banking Labs L5, L9, and L13
Technology Stack: Java 21, Spring Boot 3.3.2, Spring Security 6.3, JJWT 0.12.6, Hibernate, MySQL, H2, Springdoc OpenAPI 3.0
Package Version: 1.0.0-RELEASE | **Test Status:** 45/45 Tests Passing (100%)

---

1. Executive Summary & Domain Scope

In modern core banking architectures, a **Product** is a financial blueprint or contract template. Accounts (such as Fixed Deposit accounts, Savings accounts, or Personal Loan accounts) are never created as ad-hoc, freeform records; they are strictly **instantiated from a defined Product**.

The **Product & Pricing Module** acts as the central catalog and governance engine for these products. It encapsulates:
• Product identification and eligibility criteria (age, minimum/maximum deposit amounts).
• Allowed term ranges and interest rate bounds (rate caps, floors, base rates).
• Applicable fees, tax deduction rules, and charge schedules.
• Customer role assignments (Primary Owner, Joint Holder, Nominee).
• Tiered rate variations based on customer attributes (Senior Citizen bonus, Bank Employee concessions, Corporate Client rates).
• Complete lifecycle management (DRAFT → ACTIVE → SUSPENDED → RETIRED).

2. System Architecture & Layered Design

The module adheres to a clean, layered, hexagonal-inspired architecture designed for high maintainability, strict separation of concerns, and zero business-logic leakage:

Layer	Package / Class	Primary Responsibilities
Security Layer	com.bank.product.security • JwtAuthenticationFilter • JwtUtil • SecurityConfig	Intercepts all requests, validates HMAC-SHA256 tokens, establishes stateless SecurityContext with authorities (ROLE_ADMIN, ROLE_BANK_OFFICER), and handles 401/403 exceptions with RFC 7807 JSON.
REST Controller Layer	com.bank.product.controller • ProductController • AuthController	Exposes RESTful endpoints, enforces method-level authorization (@PreAuthorize), validates request bodies (@Valid), documents endpoints with Swagger/OpenAPI annotations.
Business Service Layer	com.bank.product.service • ProductService • ProductServiceImpl • FdCalculatorService	Executes transactional logic (@Transactional), enforces all 11 core banking business rules, manages Redis/Caffeine cache eviction, resolves customer category rate variations, and calculates FD maturity schedules.
Data Access Layer	com.bank.product.repository • ProductRepository • ProductSpecifications	Spring Data JPA abstraction, dynamic multi-predicate queries (criteria builder for status, date range, product type), and automatic optimistic locking / audit column population.

3. Relational Data Model & Entity Schema

The schema utilizes a normalized relational design with cascading 1-to-many relationships to preserve audit integrity and prevent orphaned child configurations:

Table Name	Key Columns & Constraints	Business Description
products	PK: product_id UK: product_code Enum: status, interest_type Audit: ooo_user_id, updated_by	Core product entity capturing product name, currency, effective/end validity dates, min age requirements, deposit boundaries, and active status.
product_business_rules	PK: rule_id FK: product_id → products	Defines valid deposit tenures (minTerm, maxTerm in months) and interest rate boundaries (minRate, maxRate, baseRate).
product_charges	PK: charge_id FK: product_id → products	Fee schedules including tax deductions (TDS), early withdrawal penalties, and maintenance charges.
product_roles	PK: role_id FK: product_id → products	Allowed account relationship roles (OWNER, CO_OWNER, NOMINEE) permissible during account opening.
product_rate_variations	PK: (product_id, customer_category) FK: product_id → products	Rate markups or discounts applied based on customer demographics (e.g. +1.0% for Senior Citizens, +0.5% for Employees).
fd_calculations	PK: id (BIGINT AUTO_INCREMENT)	Immutable audit trail of all maturity calculations executed by the system for compliance and reconciliation.

Standardized Enterprise Audit Columns (Regulatory Compliance Block)

To comply with core banking trace standards, entities capture a strict 11-column audit block:

Audit Column	Data Type	Constraints & Defaults	Banking Purpose
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation timestamp
updated_at	TIMESTAMP	ON UPDATE CURRENT_TIMESTAMP	Last modification timestamp
ldbid	INT	NOT NULL, DEFAULT 1	Logical Database Identifier
ooo_acpt_ts_utc_ofst	VARCHAR(6)	NOT NULL, DEFAULT '+05:30'	UTC offset of acceptance timestamp
ooo_user_id	VARCHAR(32)	NOT NULL, from JWT	Authenticated user / operator ID
ooo_ws_id	VARCHAR(32)	NOT NULL, DEFAULT 'DEFAULT'	Workstation / terminal identifier
rule_system_id	VARCHAR(32)	NOT NULL, DEFAULT 'RULES_SYS'	Business rules engine identifier
uuid	VARCHAR(36)	NOT NULL, UUIDv4	Globally unique record UUID
ooo_acpt_ts	DATETIME(6)	NOT NULL, Microsecond precision	Central acceptance timestamp
ooo_host_ts	DATETIME(6)	NOT NULL, Microsecond precision	Host server processing timestamp
ooo_local_ts	DATETIME(6)	NOT NULL, Microsecond precision	Branch/client local timestamp

4. Security Architecture & RBAC (Lab L9 Integration)

Authentication is completely stateless via JWT tokens signed with HMAC-SHA256. Role segregation is enforced at both filter and method invocation levels:

Role Name	Permitted Operations	Restricted Operations

ROLE_ADMIN	• Create Products (POST /api/product) • Soft-Retire Products (DELETE /api/product/{c}) • Update Products (PUT /api/product/{c}) • Query Products, Rates, & Defaults	None. Unrestricted administrative authority.
ROLE_BANK_OFFICER	• Update Product Configuration (PUT /api/product/{c}) • Validate FD Criteria (GET /api/product/{c}/validate) • Calculate Customer Rate (POST /api/product/{c}/rate) • View Active Catalogs & Defaults	• Cannot Create Products (403 Forbidden) • Cannot Retire Products (403 Forbidden)
ROLE_CUSTOMER / Anonymous	• Request Test Tokens (POST /api/auth/token) • Health & Info Actuator Checks	• Blocked from all Product management and operational APIs (401 Unauthorized or 403 Forbidden)

5. Core Banking Business Rules (Service Engine)

The service engine strictly enforces 11 banking validation rules before any persistence or calculation event:

- **Rule 1 (Unique Product Code):** Duplicate codes are rejected prior to persistence with HTTP 409 Conflict.
- **Rule 2 (Positive Term Limits):** minTerm <= maxTerm, and both must be strictly greater than 0 months.
- **Rule 3 (Non-Negative Rate Range):** minRate <= maxRate, and both rates must be >= 0.00%.
- **Rule 4 (Deposit & Withdrawal Limits):** minDeposit <= maxDeposit and minWithdrawal <= maxWithdrawal.
- **Rule 5 (Effective Date Range):** effectiveDate must be earlier than or equal to endDate.
- **Rule 6 (Interest Type Alignment):** Deposit products (FD, SAVINGS) must use CREDIT interest; loan products must use DEBIT.
- **Rule 7 (Minimum Age Eligibility):** Account opening applicant age must meet or exceed the product's minAge requirement.
- **Rule 8 (Active Status & Temporal Validity):** Accounts can only be instantiated against products with status ACTIVE within valid effective dates (HTTP 422 PRODUCT_INACTIVE if invalid).
- **Rule 9 (Soft Retirement):** Products are never purged from the database; retiring a product marks its status as RETIRED to protect ledger integrity.
- **Rule 10 (Contract Integrity on Updates):** In-flight updates cannot compromise existing active account bounds.
- **Rule 11 (Standard Defaults):** Provides standardized default values (tenure, opening deposit, allowable roles) for reliable wizard pre-filling.

6. FD Module Integration APIs (Lab L13)

The Fixed Deposit wizard and maturity calculation subsystems interface with the Product catalog through four specialized endpoints:

Endpoint	Method & Parameters	Behavior & Contract
Product Validation	GET /api/product/{c}/validate Query: termMonths, amount, age	Validates that product is active, tenure falls in [minTerm, maxTerm], deposit falls in [minDeposit, maxDeposit], and applicant meets minAge. Returns 200 with validation summary or 400/422 with actionable error.
Rate Determination	POST /api/product/{c}/rate Body: {termMonths, customerCategory, age}	Resolves base interest rate, automatically assigns SENIOR_CITIZEN if age >= 60, looks up rate modifier (+1.0%), and clamps effective rate strictly within [minRateLimit, maxRateLimit].

Active Products Catalog	GET /api/product/active Query: type=FD	Returns all non-retired, non-expired products ready for new account opening, optimized with Spring Cache (@Cacheable).
Account Defaults	GET /api/product/{c}/defaults	Returns standard recommended term (e.g. 12 months), initial deposit amount, and allowed ownership roles (OWNER, NOMINEE) to populate wizard forms.

7. Standardized Error Handling & HTTP Statuses

All exceptions are centrally intercepted by GlobalExceptionHandler and serialized into a uniform error response payload: {timestamp, status, errorCode, message, path}.

HTTP Status	Error Code	Root Cause / Scenario
400 Bad Request	VALIDATION_FAILED	Missing mandatory attributes, invalid numeric formats, negative rates/terms.
400 Bad Request	INVALID_PRODUCT	Business rule violations (minTerm > maxTerm, applicant under minAge).
401 Unauthorized	UNAUTHORIZED	Missing, expired, or tampered JWT Bearer token.
403 Forbidden	FORBIDDEN	Valid token but insufficient role (e.g., BANK_OFFICER attempting to create product).
404 Not Found	PRODUCT_NOT_FOUND	Product code does not match any record in catalog.
409 Conflict	DUPLICATE_PRODUCT	Attempt to create a product with an already existing productCode.
422 Unprocessable	PRODUCT_INACTIVE	Attempting to validate or create account on SUSPENDED, RETIRED, or expired product.

8. Automated Test Suite & Quality Verification

The codebase is validated by **44 automated tests** across 5 test suites with **100% pass rate**:

Test Suite Class	Count	Scope & Verification Focus
ProductEndToEndIntegrationTest	4 tests	Zero-mock end-to-end integration test over Spring Boot context: Admin creates product → Officer updates → FD validates criteria → Rate determined with senior citizen modifier → Admin retires → Validation correctly fails.
ProductSecurityTest	6 tests	Verifies 401 unauthenticated access, 403 unauthorized role escalation, token parsing, and role segregation between ADMIN and BANK_OFFICER.
ProductControllerTest	10 tests	WebMvc slice tests validating request parsing, HTTP status mappings, validation errors, and JSON responses.
ProductServiceTest	16 tests	Unit tests verifying all 11 core banking business rules, date range checks, rate clamping boundaries, and soft retirement.
FdCalculatorServiceTest	7 tests	Verifies FD compound interest calculations, maturity schedules, leap year handling, and monthly payout calculations.
OpenApiExportTest	1 test	Boots context and exports official OpenAPI 3.0 specification file.

9. Database Portability & Sharing Guidance

Question: Does the physical database need to be sent?
No. Physical database server files (e.g. MySQL /var/lib/mysql/ibdata1) must **never** be copied or transferred between developers. They are platform-dependent, hardware-specific, and prone to corruption. Instead, database portability is achieved entirely via standardized SQL scripts:

1. **DDL Script (schema.sql):** Automatically executed by Spring Boot on startup or executed manually in MySQL Workbench to create all normalized tables, foreign keys, and indexes.
2. **DML Seed Script (insert_dummy_data.sql):** Contains representative banking products (`FD001`, `SAV001`, `LN001`) with associated rules, fee schedules, allowed roles, and rate variations.
3. **Embedded In-Memory Mode (H2 Database):** When executed in test profile or standalone evaluation, the application requires zero external database installation, starting instantly in-memory.

10. Deliverables & Integration Artifacts

The following artifacts are bundled and ready for distribution to peer teams and grading environments:

Artifact Name	Path / Location	Description
Standalone Backend Folder	banking-product-backend/	Self-contained project directory with source code, test suites, Maven wrapper, SQL scripts, and documentation.
Ready-to-Send Zip Archive	banking-product-backend.zip	Clean, portable zip archive (~85 KB) ready to email, Slack, or upload. Excludes bulky build binaries and target folders.
OpenAPI 3.0 Specification	openapi.json	Complete OpenAPI / Swagger 3.0 contract definition ready to import into Swagger Editor or API gateways.
Postman Collection	postman_collection.json	20 production-ready requests with automated pre-request scripts that obtain and manage JWT tokens seamlessly.
Automated Smoke Test	smoke_test_endpoints.sh	Automated bash verification script testing authentication, product creation, updates, and FD integration calls.
Technical Documentation	PRODUCT_AND_PRICING_MODULE_EXPLANATION.md / .pdf	Comprehensive architectural reference available in both Markdown and publication-quality PDF formats.